

Image Description using Vision–Language Model (BLIP-2)

1. Introduction

Vision–Language Models (VLMs) combine computer vision and natural language processing to understand images and generate meaningful text.

This project uses **BLIP-2 (Bootstrapping Language–Image Pre-training)**, a powerful pre-trained VLM, to automatically generate captions from images.

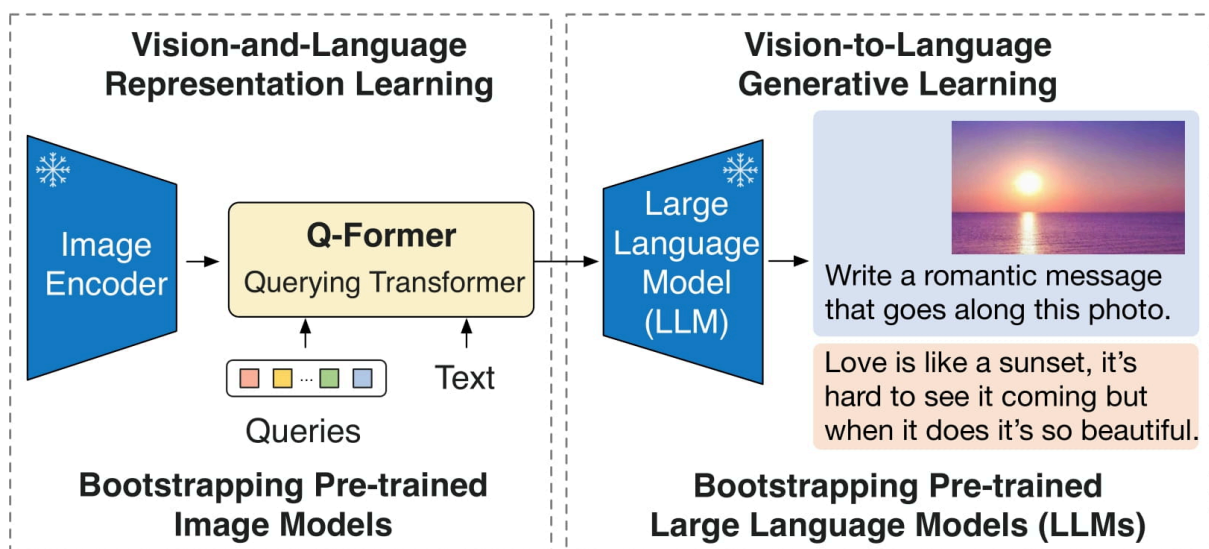
The project is implemented in Python, uses HuggingFace Transformers, and provides an interactive UI using Gradio.

Users can upload an image, and the model generates a natural-language description.

2. Model Overview — How BLIP-2 Works

BLIP-2 is a two-stage architecture designed to connect vision models (image understanding) and language models (text generation) efficiently.

BLIP-2 Architecture



2.1 Components of BLIP-2

1. Vision Encoder (e.g., ViT-L):

- Converts image → visual embeddings.
- Learn important features like objects, textures, scene layout.

2. Q-Former (Querying Transformer):

- A lightweight transformer.
- Bridges the gap between visual embeddings and language models.
- Extracts only the *important* image features and feeds them to the language model.

3. Language Model (Flan-T5 XL):

- Takes Q-Former output and generates descriptions in natural language.
- Responsible for grammar, vocabulary, sentence correctness.

2.2 Working Pipeline

Image → Vision Encoder → Q-Former → Flan-T5 → Text Caption

2.3 Why BLIP-2 is Powerful

- Does not require expensive end-to-end training.
- Works with large language models like T5, Flan-T5, or OPT.
- Produces accurate captions even with complex scenes.

3. Input and Output

Input

- Any image uploaded through the Gradio interface.
- Format: JPG, PNG, JPEG
- Type: PIL Image

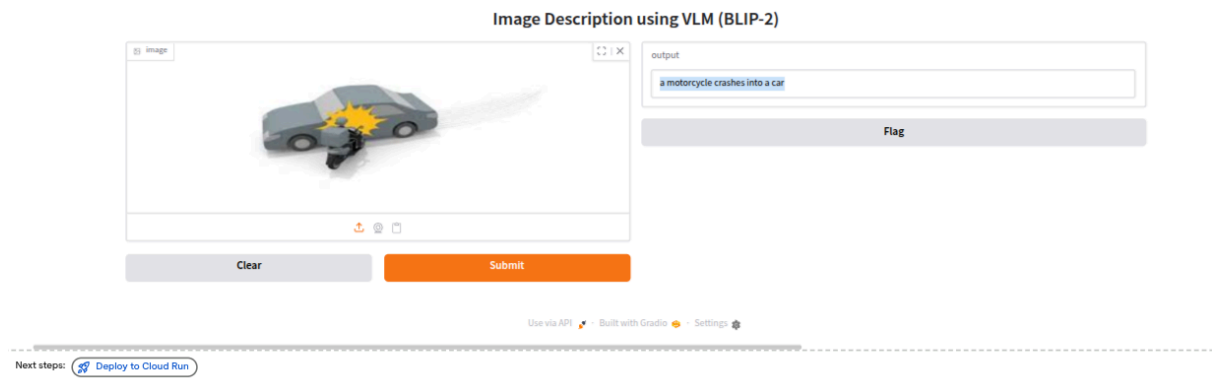
Output

- A natural language caption describing the image.
- Example:
"A dog running through a grassy field with a ball."

Input:



Output: a motorcycle crashes into a car



4. . Full Workflow of the Project

Step-by-step Pipeline

1. The user uploads an image.
2. The image is processed using Blip2Processor:
 - Resizing
 - Normalization
 - Conversion to tensors
3. The model generates output tokens using `model.generate()`.
4. Output tokens are decoded back to text description.
5. The generated text is displayed through Gradio UI.

5. Code Explanation:

5.1 Installing dependencies

```
!pip install transformers accelerate pillow gradio
```

Installs:

- **transformers** → **BLIP-2 model**
- **accelerate** → **Device optimization (GPU/CPU)**

- `pillow` → Image handling
- `gradio` → Web UI

5.2 Import Libraries

```
from transformers import Blip2Processor,
Blip2ForConditionalGeneration
from PIL import Image
import torch
import gradio as gr
```

- Imports BLIP-2 processor & model
- PIL for working with images
- Torch for GPU execution
- Gradio for UI

5.3 Load the BLIP-2 Processor and Model

```
processor =
Blip2Processor.from_pretrained("Salesforce/blip2-flan-t5-xl")
model = Blip2ForConditionalGeneration.from_pretrained(
    "Salesforce/blip2-flan-t5-xl",
    torch_dtype=torch.float16,
    device_map="auto"
)
```

Processor

- Takes raw image
- Applies resizing, tensor conversion
- Prepares data in correct format for BLIP-2

Model

- Loads the BLIP-2 (Flan-T5 XL) model
- `torch.float16` → Faster inference, less memory
- `device_map="auto"` → Auto-select GPU/CPU

5.4 Caption Generation Function

```
def describe(image):
    inputs = processor(image, return_tensors="pt").to("cuda")
    output = model.generate(**inputs)
    caption = processor.decode(output[0], skip_special_tokens=True)
    return caption
```

Breakdown

1. `processor(image)`
Converts image → processed tensors.
2. `to("cuda")`
Moves the tensors to GPU.
3. `model.generate()`
Generates caption tokens from the image.
4. `processor.decode()`
Converts tokens → natural language sentence.
5. Return caption
Final result.

5.5 Gradio User Interface

```
interface = gr.Interface(
    fn=describe,
    inputs=gr.Image(type="pil"),
    outputs="text",
    title="Image Description using VLM (BLIP-2)"
)
```

- `fn=describe` → Function called when user uploads an image.
- `inputs=gr.Image(type="pil")` → Accepts image input.
- `outputs="text"` → Displays generated captions.
- Title is shown at the top of the UI.

5.6 Launch the App

```
interface.launch()
```

- Starts a local or public Gradio server.
- UI allows the user to upload images and get captions.

6. Advantages

- Works with any image
- Uses state-of-the-art VLM (BLIP-2)
- Fast GPU-supported caption generation
- Accessible via Gradio interface
- Useful for visually impaired applications
- Can be extended for image-question answering
- Can be integrated into websites or mobile apps

7. Reference

https://huggingface.co/docs/transformers/en/model_doc/blip-2
https://huggingface.co/docs/transformers/en/model_doc/blip-2#transformers.Blip2ForConditionalGeneration

<https://medium.com/@femiloyeseun/blip-2-a-breakthrough-approach-in-vision-language-pre-training-1de47b54f13a>